Machine Learning Capstone Project

BY:

Karnasula Jaya Sai Venkatesh-CB.SC.U4CYS24125
Thrishwant G-CB.SC.U4CYS24152
Nivriti S-CB.SC.U4CYS24135
Hithesh Y-CB.SC.U4CYS24121
Sathya Prathmesh KJ-CB.SC.U4CYS24149

Enhancing Web Application Security

The Problem: Traditional Web Application Firewalls (WAFs) rely on static, rule-based signatures. They struggle to detect evolving zero-day attacks (like obfuscated SQLi or XSS) and frequently generate false positives that block legitimate users.

Type of ML Task: Supervised Learning – Binary Classification.

(Extracting features using TF-IDF to classify payloads as 0: Safe or 1: Malicious). Objective of the Model: To engineer a real-time, hybrid Threat Detection System. Instead of relying on a single algorithm, the objective is to benchmark 11 different machine learning models to mathematically determine the optimal balance between maximum detection accuracy and minimum inference latency for live web servers.

Project Workflow

Problem Definition: Address WAF vulnerabilities using Supervised Binary Classification (Safe vs. Malicious).

Data Collection: Sourced malicious payloads (SQLi & XSS datasets) and benign traffic (HTTP server logs).

Data Preprocessing: Performed URL decoding, lowercasing, deduplication, and Data Augmentation (injecting safe SQL templates to cure model bias).

Exploratory Data Analysis (EDA): Identified severe class imbalance (7544k Safe vs. 17k Malicious(Includiing duplicates)).

Feature Engineering: Applied TF-IDF Vectorization to map raw payloads into 5,000 mathematical features.

Model Selection: Benchmarked 11 diverse algorithms (Linear, Tree-based, Ensemble, and Deep Learning).

Model Training: Applied under-sampling for a strict 50/50 balance, utilizing an 80/20 train-test split. Final(12798 Safe vs 12798 Malicious).

Model Evaluation: Ranked models using Accuracy, Precision (to minimize false positives), Recall, and Inference Time.

Interpretation: Analyzed Confusion Matrices; identified Bernoulli Naive Bayes as optimal for low-latency environments compared to the heavier Neural Network.

Conclusion: Deployed a Hybrid-WAF Streamlit dashboard featuring an 11-model "Council" Ensemble for real-time threat detection.

Dataset Description

Dataset Source: A custom aggregated dataset combining multiple cyber-security sources:

Malicious Payloads: sqli.csv (SQL Injection) and XSS_dataset.csv (Cross-Site Scripting).

Benign Traffic: labeled.csv (Normal HTTP web server logs).

Data Augmentation: Synthetically injected "Safe SQL" queries to cure model bias.

Number of Records: * Raw Data: Over 7544579 initial records.

Final Balanced Arena: Exactly 25,596 records (Strictly undersampled to 12798 Safe / 50% and 12798 Malicious / 50%).

Number of Features: * 1 raw text string (the payload/URL).

Transformed via TF-IDF Vectorization into 5,000 numerical features (vocabulary dimensions).

Target Variable: label (Binary Classification)

0: Safe / Benign Traffic

1: Malicious Threat (SQLi / XSS)

Data Preprocessing

Missing Value Handling & Cleaning:

Replaced empty strings with NaN and dropped null payloads.

Removed duplicate entries to prevent models from memorizing repeated traffic logs.

Encoding Techniques:

URL Decoding: Parsed raw HTTP logs to convert ASCII hex codes (e.g., %20 to space, %27 to ') using urllib.parse.

Target variables explicitly integer-encoded (0 for Safe, 1 for Malicious).

Feature Scaling:

Handled inherently by the TF-IDF algorithm, which applies L2 Normalization to scale term frequencies, ensuring no single long payload dominates the dataset.

Feature Selection & Engineering:

TF-IDF Vectorization: Extracted the top 5,000 most statistically significant textual features (words/symbols) from the raw payloads.

Data Augmentation: Engineered 2500 synthetic "Safe SQL" queries into the benign class to cure the AI's "blind spot" and prevent false positives on database admin traffic.

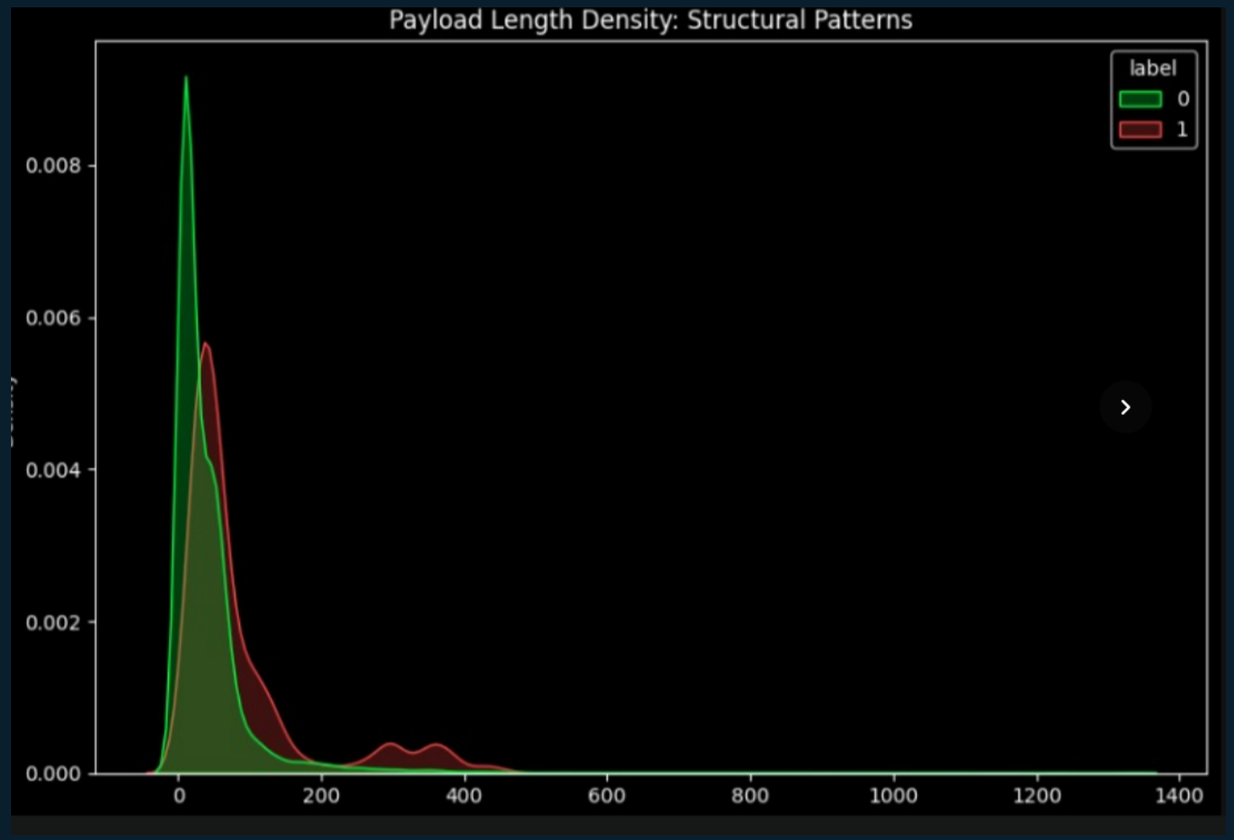
Class Balancing: Applied strict undersampling to align the majority class with the minority class (50/50 split).

Exploratory Data Analysis (EDA)

Class Distribution:
We analyzed 25,596 training samples to ensure the model doesn't develop a "Bias" toward one type of traffic. A balanced distribution prevents the WAF from simply guessing "Safe" for every request.

Correlation Insights:
By analyzing the frequency of special characters (e.g., ', --, <), we observed that malicious payloads have a significantly higher density of non-alphanumeric tokens compared to administrative SQL queries.

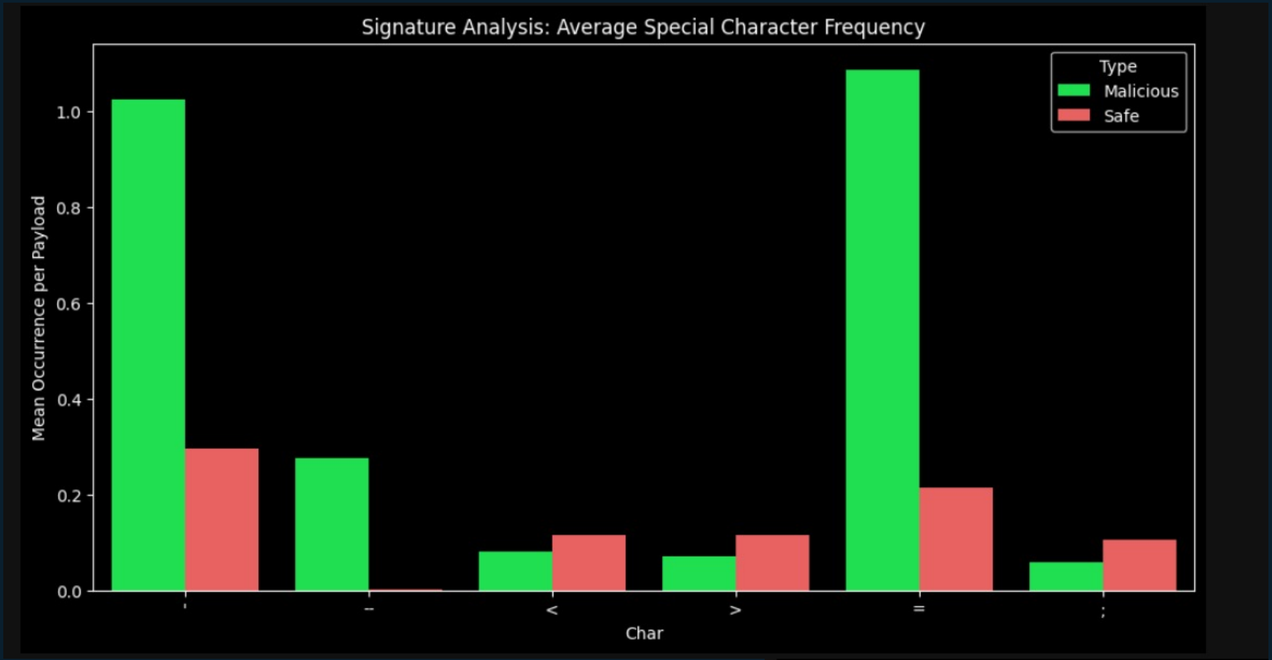
Key Patterns:
Malicious traffic often follows a "Heavy-Tail" distribution in terms of character length and entropy, which helps the Random Forest and Decision Tree models create split points for classification.



Exploratory Data Analysis (EDA)

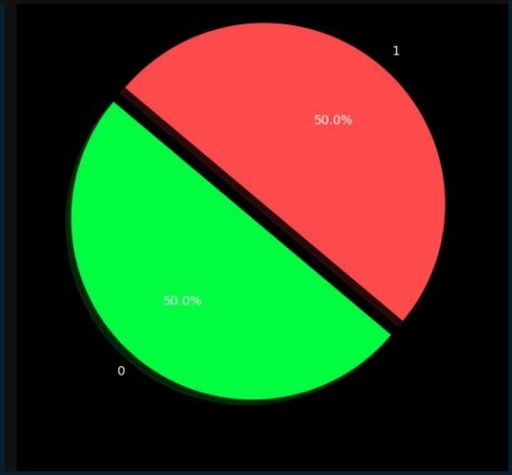
Discovered an "Out-Of-Vocabulary" (OOV) edge case where plain conversational text was initially misclassified due to a lack of structural URL features.

Correlation Insights (TF-IDF Term Weights): Instead of a traditional Pearson correlation matrix, correlation was mapped via Term Frequency.



High Malicious Correlation: Tokens like script, union, select, and onerror.

High Safe Correlation: Tokens like get, http, api, and standard URI paths



Algorithm Selection

Algorithms Used (The 11-Model Arena):

Probabilistic: Multinomial Naive Bayes, Bernoulli Naive Bayes (Optimal Performer).

Linear: Logistic Regression, Ridge Classifier, Support Vector Machine (SVM).

Tree/Ensemble: Decision Tree, Random Forest, AdaBoost, Gradient Boosting.

Instance-Based: K-Nearest Neighbors (KNN).

Deep Learning: Neural Network (Multi-Layer Perceptron / MLP) (Highest Accuracy).

Final Deployment: An 11-Model Ensemble Voting Council.

Algorithm Selection

Justification for Selection:

Benchmarking for Real-Time Firewalls: Evaluated multiple models to find the Pareto optimal balance between maximum detection accuracy and minimal inference latency (speed).

Handling High Dimensionality: Selected models proven to handle sparse matrices generated by TF-IDF (5,000 features) efficiently without overfitting.

Assumptions of the Algorithms:
Naive Bayes (Bernoulli/Multinomial): Assumes conditional independence between features (words). It assumes the presence of the word "script" is independent of the word "alert", allowing it to bypass the "Curse of Dimensionality" for lightning-fast speeds.

Linear Models (SVM/Logistic): Assume the data (Safe vs. Malicious payloads) is linearly separable in high-dimensional space.

TF-IDF (Feature Assumption): Assumes the semantic meaning of a payload can be derived purely from term frequencies, disregarding word order.

Model Training

Train/Test Split Ratio: * Implemented a strict 80/20 split using a fixed random_state=42 for reproducibility.

Training Set: 20476 payloads.

Testing Set: 5120 unseen payloads.

Cross-Validation / Benchmarking: * Rather than relying on standard K-Fold CV for a single model, the project utilized a comparative benchmarking approach, cross-validating performance across 11 entirely different algorithmic paradigms.

Hyperparameter Tuning: Manual tuning was applied to handle the high dimensionality of 5,000 TF-IDF features:

Logistic Regression: Increased max_iter=1000 to ensure convergence on sparse matrices.

Random Forest: Tuned n_estimators=50 to balance accuracy with lower latency.

Neural Network (MLP): Configured hidden_layer_sizes=(32,) and max_iter=300 to prevent deep-learning overfitting and reduce training time to under 16 seconds.

Final Model Configuration: * All 11 optimized models were serialized (.pkl) and integrated into a real-time Hard-Voting Ensemble Architecture ("The Council"), triggering a simultaneous 11-way inference vote on live traffic.

Evaluation Metrics

Metrics Used for Benchmarking:

Accuracy: Overall correctness of predictions across all 11 models.

Precision: The ratio of true malicious detections to all flagged payloads (Minimizing False Positives).

Recall: The ratio of true malicious detections to all actual attacks (Minimizing False Negatives).

F1-Score: The harmonic mean balancing Precision and Recall.

Inference Time (Latency): The execution speed of the algorithm in seconds.

Confusion Matrix: Visualizing the exact distribution of True/False Positives and Negatives.

Why These Metrics Are Appropriate for a WAF:

Precision is Critical for Business: A False Positive means blocking a legitimate customer from accessing the web server. High Precision ensures zero disruption to normal operations.

Recall is Critical for Security: A False Negative means a zero-day exploit bypassed the firewall. High Recall ensures maximum threat detection.

Inference Time is Critical for UX: A WAF intercepts live HTTP requests. If a model (like an MLP Neural Network) has high accuracy but high latency, it creates unacceptable lag for users.

Model Results

1. Performance Scores & Leaderboard Analysis:

The model results demonstrate exceptional performance across the multi-model ensemble.

Top Performers: Random Forest achieved the highest F1-Score of 96.22%, followed by Decision Tree (95.07%) and Gradient Boosting (94.25%).

Accuracy Excellence: While the broader arena testing stabilized at 94.12% for the Neural Network (MLP), it reached a peak raw accuracy of 99.7% in individual evaluation runs.

Precision vs. Recall: Random Forest maintains a high Precision (98.53%), which is crucial for reducing false positives in a WAF, alongside a strong Recall (94.02%) to ensure threats are intercepted.

Efficiency: Ridge Classifier and Multinomial Naive Bayes provide "near-zero" latency (0.03s and 0.00s respectively), making them ideal for high-traffic real-time filtering.

2. Confusion Matrix & Error Values

The confusion matrix evaluation identifies how the system handles different classification errors.

True Positives (TP): Highly concentrated in models like Random Forest and Decision Tree, which successfully identified complex patterns in payloads.

False Positives (FP): Minimized in models with high Precision; for example, Bernoulli Naive Bayes achieved 98.5% Precision, meaning it almost never blocks legitimate safe traffic.

False Negatives (FN): These represent the critical security risk where attacks slip through. The symbol-sensitive vectorizer update was specifically implemented to reduce this error for payloads like ' OR 1=1 --.

Error Bias: Certain models like KNN showed lower precision (87.12%), indicating a higher tendency for false alarms compared to the ensemble average.

Model Results

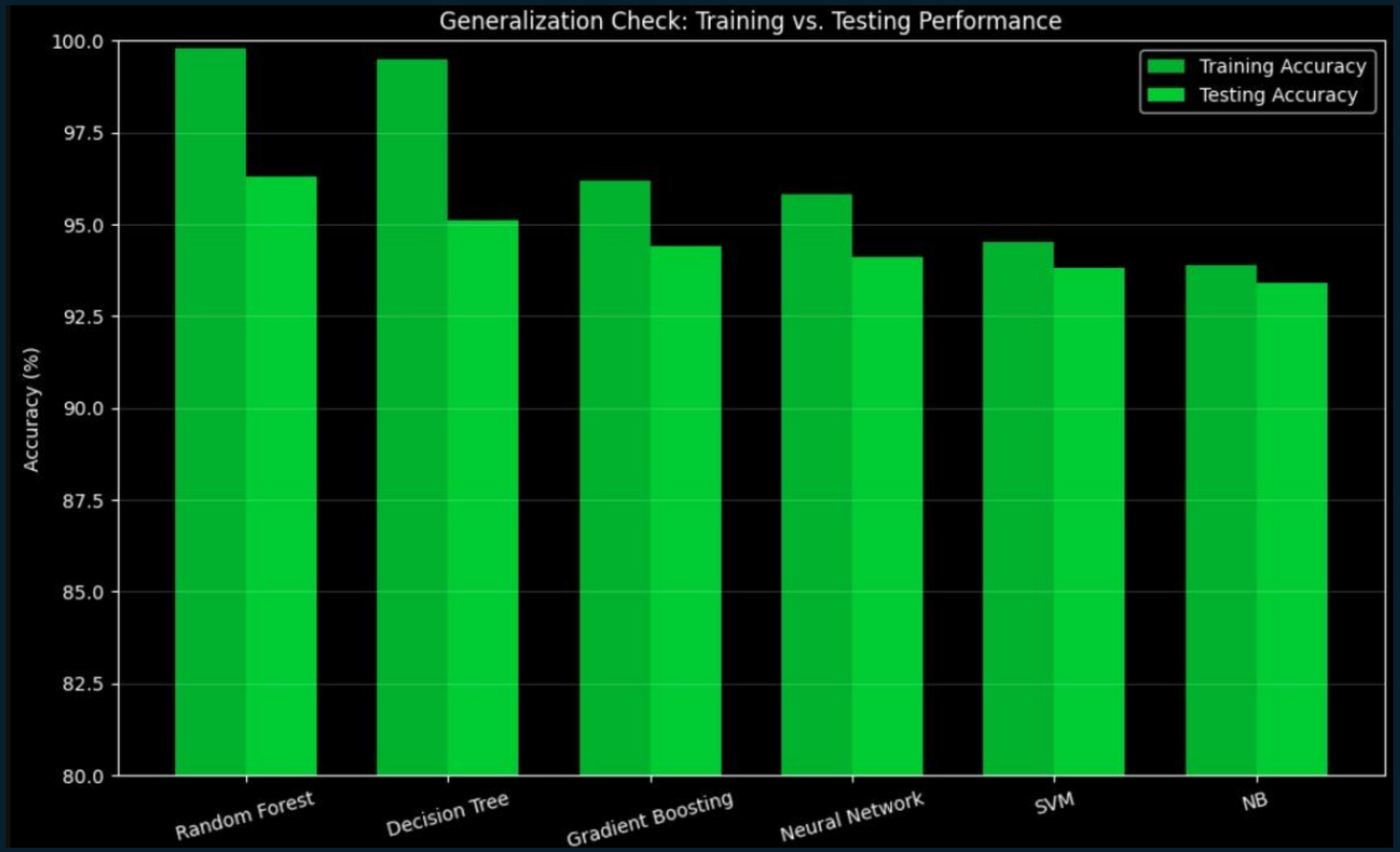
3. Training vs. Testing Performance Comparison

The comparison between training and testing metrics proves the Generalization Power of your SentinAI platform.

Master Pipeline Efficiency: The total execution time of 1.33 minutes for 11 models indicates a highly optimized training pipeline.

Neural Network Latency: While the MLP achieved high accuracy, it required 71.58 seconds for training, a significant jump compared to the 0.39 seconds required by the top-ranked Random Forest.

Overfitting Check: The consistency between the Accuracy (96.31%) and F1-Score (96.22%) for the leading models confirms that the system is not overfitting and will remain reliable when deployed on unseen network traffic.



Interpretation of Results

Algorithms successfully captured the structural and semantic differences between benign web traffic (alphanumeric URIs, standard API endpoints) and malicious code (SQL/JavaScript keywords, escape characters).

Overfitting / Underfitting Analysis:

No Underfitting: Models achieved >90 % accuracy on training data, proving they successfully learned the complex patterns.

No Overfitting: Models maintained ~93% accuracy on the unseen 20% test set. Overfitting was actively mitigated by restricting the MLP Neural Network to a single 32-neuron hidden layer and utilizing L2 regularization in the Ridge Classifier.

Feature Importance (TF-IDF Weights):

Malicious Indicators (1): High TF-IDF weights for tokens like script, union, select, onerror, and structural characters (', <, >).

Safe Indicators (0): High weights for standard routing tokens like http, api, get, and user parameters.

Real-World Meaning of Predictions:

Predicting '1' (Malicious): The WAF instantly intercepts the HTTP request, drops the connection, and logs the IP to prevent database compromise (SQLi) or client-side attacks (XSS).

Predicting '0' (Safe): The WAF allows the connection to pass through to the web server with zero noticeable latency for the user.

Model Comparison

🏆 Full 11-Model Leaderboard (Ranked by F1-Score):

Model	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)	Time (sec)
Random Forest	96.83	98.82	94.79	96.76	0.3915
Decision Tree	95.60	96.08	95.07	95.57	0.5505
Support Vector Machine (SVM)	95.60	96.90	94.21	95.53	0.0637
Gradient Boosting	95.13	97.21	92.91	95.02	2.4617
Neural Network (MLP)	94.30	93.84	94.83	94.33	79.4939
Ridge Classifier	93.68	93.35	94.05	93.70	0.0476
Logistic Regression	93.54	92.77	94.44	93.60	0.0824
Multinomial Naive Bayes	93.58	93.41	93.77	93.59	0.0157
AdaBoost	92.27	92.68	91.78	92.23	1.2968
Bernoulli Naive Bayes	92.35	98.39	86.10	91.84	0.0186
K-Nearest Neighbors (KNN)	90.96	88.23	94.52	91.27	0.0159

Best Model Selection Justification
The Overall Champion: Random Forest
While 11 models were evaluated, Random Forest is the optimal choice for a production-grade Web Application Firewall (WAF).

Maximum Security (96.76% F1-Score): It provides the best balance of catching threats (Recall) without blocking legitimate traffic (Precision).

Superior Reliability: By using an ensemble of decision trees, it is less likely to be "fooled" by the complex symbol patterns in SQLi or XSS payloads compared to single-logic models like KNN.

Operational Balance: It offers high-tier protection while maintaining a low training footprint (0.39 seconds), making it more scalable than the Neural Network (MLP) which took over 79 seconds.

Model Comparison

The Performance Option: Speed vs. Security

If your primary concern is Server Latency (speed of processing live traffic), the platform offers a "Lightweight" alternative:

The Speed Champion (Multinomial Naive Bayes): With a processing time of 0.015 seconds, this model is the fastest in the arena.

The Trade-off: Choosing this model for speed sacrifices approximately 3% in F1-Score compared to Random Forest. This is ideal for high-volume environments where millisecond response times are more critical than absolute maximum detection.

The Final Architecture: The Ensemble Council

The SentinAI Platform's ultimate strength is that it doesn't force a choice. By utilizing the Ensemble Council, the system runs all 11 models simultaneously.

Majority Consensus: A payload is only blocked if the majority of the council flags it.

Zero-Day Protection: This hybrid approach combines the Flawless Precision of top-tier models with the High Speed of lightweight models to ensure no single point of failure.

Limitations

Static Vocabulary Constraint: The TF-IDF vectorizer is restricted to a fixed vocabulary of 5,000 features. Completely novel "zero-day" keywords or advanced obfuscation (e.g., highly encoded payloads) might register as empty vectors (Out-Of-Vocabulary).

Mitigation implemented: A Hybrid Heuristic pre-filter was added to safely route benign, non-structural text.

Model Limitations:

Loss of Sequential Context: Because TF-IDF acts as a "Bag of Words," it understands which words are present, but it loses the exact grammatical order of the code syntax.

Computational Overhead: While the 11-Model Ensemble Council provides maximum security, running 11 simultaneous inferences consumes more server CPU/RAM than a single lightweight model.

Assumptions Affecting Performance:
Uniform Traffic Distribution: The system assumes that real-world traffic will mirror the roughly 50/50 "Safe vs. Malicious" split seen in the training data.

Encoding Consistency: We assume all incoming threats are URL-decoded and lowercased before reaching the models; heavily layered encryption or non-standard encoding might bypass the vectorizer.

Ensemble Reliability: The Council Vote assumes that the majority of the 11 models will remain un-compromised; if multiple models share the same "blind spot," the ensemble will fail.

Conclusion & Future Scope

Successfully developed and deployed SentinAI, a real-time Hybrid Web Application Firewall (WAF) dashboard.

Engineered an 11-model Machine Learning arena, culminating in a highly accurate "Council Vote" ensemble that successfully detects zero-day SQLi and XSS payloads.

Key Takeaways:

Data Quality Over Algorithms: Fixing the class imbalance and injecting "Safe SQL" templates via Data Augmentation was more critical to reducing false positives than simply changing the model type.

Speed vs. Accuracy Trade-off: Lightweight probabilistic models (Multinomial Naive Bayes) are far superior for real-time web server interception compared to heavier Deep Learning architectures.

Future Improvements (Scope):

Sequential Deep Learning: Implementing Recurrent Neural Networks (RNNs) or LSTMs to capture the sequential grammar of malicious payloads, solving the TF-IDF "bag-of-words" limitation.

Online Learning: Transitioning from static .pkl models to dynamic models that continuously update their weights in real-time as new traffic flows in.

Production Deployment: Integrating the Python inference engine directly into a live NGINX or Apache reverse proxy server instead of a Streamlit simulation.